

ServiceSetu

End-to-end test & security report

Generated 02 October 2026, 12:36 from a live run against the development stack

Area	What was checked	Run	Result
Backend suite	Unit and integration tests across 19 suites	299	299 passed
Security probes	Live attacks over HTTP against the running API	32	32 blocked
API surface	Every registered route checked for guards and 5xx	118	0 unguarded
Frontend walk	Every page per role, console errors captured	44	44 clean

All 493 checks passed. No failures.

Backend 64160 ms. Probes and route sweep run against localhost:5000; frontend walked in a real browser against localhost:5175.

Scope and method

Four independent layers were exercised. The backend suite runs against an isolated `servicesetu_test` database so it can create and destroy data freely. The security probes and route sweep run over HTTP against the development server, with no internal imports, so they see only what a client sees. The frontend walk drives a real browser, signing in as each role and visiting every page while recording console errors. Test data created during the run was removed afterwards and the database verified clean.

1. Backend test suite

299 tests across 19 suites, covering the domain from schema constraints through to money movement. Every suite passed.

Suite	Time	Result
Multi-service bookings	1092 ms	pass
Phase 10: ratings and reviews	1274 ms	pass
Phase 11: governance and analytics	2865 ms	pass
Phase 12: hardening and delivery	199 ms	pass
Phase 13: agencies	3863 ms	pass
Phase 1: application surface	138 ms	pass
Phase 2: schema integrity	259 ms	pass
Phase 3: authentication and role flow	1347 ms	pass
Phase 4: catalogue and provider profile	563 ms	pass
Phase 4: provider self-service	1339 ms	pass
Phase 5: verification and KYC	860 ms	pass
Phase 6: discovery and search	650 ms	pass
Phase 7: booking and scheduling	1152 ms	pass
Phase 7: lifecycle, policy and scope	1043 ms	pass
Phase 8: job status tracking	1302 ms	pass
Phase 8: live status stream	1433 ms	pass
Phase 9: payments and invoicing	1738 ms	pass
Razorpay driver	6 ms	pass
Verification codes go by email	931 ms	pass

What the suites assert

Schema integrity — a provider cannot be double-booked for an overlapping slot, a cancelled booking releases its slot, a completed booking must record what it charged, and every status change writes a history row.

Money — commission is frozen per booking so a later settings change never rewrites history; a payout cannot pay the same earnings twice; refunds move through the same ledger as everything else.

Access — role boundaries, ownership checks, and the rule that contact details reach the other party only once a booking is accepted.

Multi-service bookings — the newest suite, covering the feature added in this cycle.

2. Multi-service booking (new this cycle)

A customer can book several services in one visit. These assert the arithmetic rather than that the request merely succeeded, because the arithmetic is the feature.

#	Case	Why it matters	Result
1	Total is the sum of the chosen services	Two services priced separately bill as one total	pass
2	Reserved slot is long enough for both jobs	Duration is summed, so the calendar is protected	pass
3	Call-out fee is charged once, not per service	One trip means one visit charge	pass
4	Commission is weighted by what each service costs	A dearer item carries proportionally more	pass
5	Headline category is the service chosen first	Keeps reports and indexes working unchanged	pass
6	Each service keeps its own category rate	A mixed-rate booking settles each at its own rate	pass
7	The same service twice is collapsed	A duplicate tick is a slip, not a quantity	pass
8	A service the provider does not offer is refused	Named, so the customer knows which to untick	pass
9	An empty service list is refused	Validation rejects a booking with nothing in it	pass
10	The older single-service shape still works	Backwards compatible with any existing client	pass
11	Rescheduling keeps the full duration	Regression: moving a booking must not shrink its slot	pass
12	The provider sees every service on the job	The job sheet lists what they are turning up to do	pass
13	Completion ceiling covers every service	Regression: a two-service job can be completed	pass
14	A finished job shows no step in progress	Regression: the timeline stops saying "happening now"	pass
15	Report rows name every service	Regression: admin reports list both, not just the first	pass
16	Slots can be sized for the whole visit	Times offered are times that can actually be booked	pass

Four defects were found and fixed during this cycle, all caught by testing rather than review: the completion ceiling read one service's cap and blocked completion outright; the progress timeline marked a finished job as still happening; the invoice header named only the first service; and admin report rows did the same. Each now has a regression test that was verified to fail with its fix reverted.

3. Security probes

Live attacks over HTTP against the running API. Each probe states the attack, then asserts the defence. All 32 were blocked.

Area	Attack attempted	Expected	Result
Access control (IDOR)	Request a booking that does not exist	404	blocked
Access control (IDOR)	A provider not assigned to the booking requests it directly	403 or 404	blocked
Access control (IDOR)	A provider not assigned to the booking tries to accept it	403 or 404	blocked
Audit trail	Re-save the commission setting as admin	200	blocked
Authentication	Call /bookings with no Authorization header	401	blocked
Authentication	Send a garbage token	401	blocked
Authentication	Self-sign a token claiming role=admin	401	blocked
Authentication	Wrong password on a real account	generic message, no hint the account exists	blocked
Authentication	Register with "password"	422	blocked
Authorisation	Customer token on an admin route	403	blocked
Authorisation	Provider token on the admin verification queue	403	blocked
Authorisation	Customer token on /earnings	403	blocked
Authorisation	Provider token on the payout batch	403	blocked
Authorisation	Customer token writing platform settings	403	blocked
Data minimisation	Inspect the public provider search payload	no password hash, email or bank details	blocked
Data minimisation	Read the payouts report as admin	account numbers masked to last four	blocked
File access	Ask for ../../.env through the file endpoint	403 or 422	blocked
File access	Invent a signature for a KYC document	403	blocked
File access	Request a document with the key alone	422	blocked
Information disclosure	Request a route that does not exist	standard error envelope, no stack	blocked
Injection	Search for ' OR 1=1--	handled safely, no 500	blocked
Input validation	Pass a non-UUID where an id is expected	422	blocked

Area	Attack attempted	Expected	Result
Input validation	Ask for 999999 results	422	blocked
Input validation	POST /auth/login with no body	4xx, not a crash	blocked
Money integrity	Book while dictating quotedAmountMinor=1	server price used, not the client one	blocked
Money integrity	Forge a payment.captured webhook	rejected (4xx)	blocked
Money integrity	Send an unsigned webhook	rejected (4xx)	blocked
Privilege escalation	Register with role=admin in the body	refused or downgraded	blocked
Privilege escalation	PATCH own profile with role=admin	role unchanged	blocked
Rate limiting	Inspect RateLimit headers	ratelimit-limit present	blocked
Secrets in responses	Register and inspect the response for the 6-digit code	no code in the body when a mail server is configured	blocked
Transport and headers	Inspect response headers	nosniff + frame options + no x-powered-by	blocked

4. API surface

Every route the application registers was read from the live Express router and probed anonymously. Two properties were asserted: nothing returns a server error to an unauthenticated caller, and every route that is not public by design demands authentication.

Measure	Count	Result
Routes registered	118	probed
Public by design	42	expected
Guarded	76	challenge anonymous
Server errors (5xx)	0	none
Unguarded private routes	0	none

Public endpoints

The 42 public routes are public because they cannot require a session: registration, sign-in, password reset and contact confirmation all happen before one exists, and the catalogue, provider profiles and their reviews are what an anonymous visitor came to read. They are defended by rate limiting and schema validation instead, both covered by the probes above.

5. Frontend

Driven in a real browser against the development server. Each role was signed in and every page in its area visited, with console errors recorded per page. Route guards were tested by attempting to reach other roles' areas.

Role	Page	Rendered	Console
anonymous	/	yes	clean
anonymous	/search	yes	clean
anonymous	/about	yes	clean
anonymous	/help	yes	clean
anonymous	/for-providers	yes	clean
anonymous	/signin	yes	clean
anonymous	/register	yes	clean
anonymous	/forgot-password	yes	clean
anonymous	/no-such-page	yes	clean
anonymous	/bookings -> /signin	yes	clean
anonymous	/account -> /signin	yes	clean
anonymous	/provider -> /signin	yes	clean
anonymous	/admin -> /signin	yes	clean
anonymous	/agency -> /signin	yes	clean
customer	/bookings	yes	clean
customer	/invoices	yes	clean
customer	/reviews	yes	clean
customer	/account	yes	clean
customer	/search	yes	clean
customer	/	yes	clean
customer	/admin -> /	yes	clean
customer	/provider -> /	yes	clean
provider	/provider	yes	clean
provider	/provider/requests	yes	clean
provider	/provider/schedule	yes	clean
provider	/provider/earnings	yes	clean
provider	/provider/profile	yes	clean
provider	/provider/verification	yes	clean

Role	Page	Rendered	Console
provider	/admin -> /provider	yes	clean
agency	/agency	yes	clean
agency	/agency/team	yes	clean
agency	/agency/jobs	yes	clean
agency	/agency/profile	yes	clean
agency	/agency/verification	yes	clean
agency	/admin -> /agency	yes	clean
admin	/admin	yes	clean
admin	/admin/reports	yes	clean
admin	/admin/verification	yes	clean
admin	/admin/categories	yes	clean
admin	/admin/bookings	yes	clean
admin	/admin/disputes	yes	clean
admin	/admin/reviews	yes	clean
admin	/admin/payouts	yes	clean
admin	/admin/settings	yes	clean

Route guards

An anonymous visitor attempting any private area is redirected to sign-in. A signed-in user attempting another role's console is redirected to their own home. Both were verified for every role, and the redirect destination recorded above.

Critical journey

The multi-service booking flow was driven end to end through the interface: two services selected (Rs 249 + Rs 1,799), the running total and summed duration confirmed as Rs 2,048 over 3 hr 30 min, the slot grid confirmed to narrow to only those windows long enough to hold the visit, the address step completed and the request sent. The resulting booking rendered as expected with both services itemised. The test booking was then deleted.

Build

Production build succeeds. All 28 design tokens resolve in the compiled CSS, and no utility class fails to compile - a check added after a class was found silently producing no rule at all, leaving an element unstyled with no error anywhere.

6. Known gaps and decisions

Stated plainly so they are decisions rather than surprises.

Item	Status	Detail
Razorpay webhook secret is unset	Open	Payments are exercised through a signed test harness. Set RAZORPAY_WEBHOOK_SECRET in backend/.env before production.
Visit fee on a multi-service booking	Decision	Charged once, at the highest of the chosen services, rather than once per service. One trip, one call-out. Awaiting sign-off.
Commission on a multi-service booking	Decision	Frozen per service at its own category rate; the booking-level rate is the price-weighted average. Awaiting sign-off.
"By service" counts bookings containing a service	By design	A two-service booking appears under both, so the column sums to more than the number of bookings. Revenue is split by price share.
WhatsApp OTP delivery	Planned	Codes currently go by email. WhatsApp Cloud API delivery is scheduled next; email should remain as the fallback.

How to reproduce this report

```
cd backend && npm test
node scripts/security-probe.mjs report.json
node scripts/api-surface.mjs surface.json
cd ../frontend && npx vite build
```

The security probe and surface sweep need the API running on port 5000. The frontend walk is driven through the browser against port 5175.